ME3 Voice Duplex: A Tool-Using, Full-Duplex Voice Agent for Raspberry Pi

Crizepvill F. Dumalaog
Mapua University
Master of Engineering in Artificial Intelligence
Manila, Philippines
202521406

Abstract

ME3 is a Raspberry Pi voice-agent prototype that combines local speech processing, full-duplex audio, a remote Qwen language-model brain, and project-owned Model Context Protocol (MCP) services. The system is designed for an important interaction beyond question answering: a user can ask by voice for a web application or other coding task, and the model can call bounded file, search, command, and background-job tools to perform the work. No task-specific model training is required. Instead, the system integrates pretrained Silero VAD, faster-whisper ASR, Piper TTS, and an OpenAI-compatible Qwen/vLLM endpoint. This paper describes the architecture, tool-call protocol, safety boundary, local verification, Raspberry Pi release workflow, and evidence boundary for the separate FULL-DUPLEX-BENCH v1.5 evaluation. The current implementation is an engineering release, not a claim of completed paper-grade overlap results.

Keywords

voice agents, full duplex, Raspberry Pi, tool calling, MCP, Qwen, speech interaction, edge computing

1 Introduction

Voice interfaces are often presented as a sequence: wait for the user to stop, transcribe one utterance, generate text, and play a response. A useful assistant must also manage interruption and action. A user should be able to say "build a web app" and receive evidence of files and verification, rather than a plausible paragraph describing code that was never created.

ME3 addresses these requirements with a local audio loop and a remote tool-using language model. The design separates four concerns. First, the Pi keeps voice-sensitive processing local through VAD, ASR, and TTS. Second, a Qwen model served by an OpenAI-compatible vLLM API performs dialogue reasoning. Third, typed tools provide an auditable action surface. Fourth, a release and deployment workflow distinguishes local preparation from actual Pi installation.

The project is intentionally not a new model-training project. The agent behavior comes from inference and orchestration: pretrained speech models, Qwen inference, MCP/tool dispatch, and a bounded local execution policy. This choice reduces iteration time and makes the engineering questions explicit.

The contributions of this work are:

- a full-duplex local voice loop with barge-in during TTS playback;
- an OpenAI-compatible Qwen tool loop that can create and verify a static web application through voice or text;
- one bounded registry shared by direct Qwen calls and MCP wrappers;
- background task supervision so builds do not block the microphone; and
- a reproducible local-dev, package, deploy, and evidence workflow for the Raspberry Pi target.

2 System Requirements and Scope

The required runtime contains four owned MCP services: VAD, ASR, TTS, and tasks. Pipecat is retained as the harness-facing integration path, while the local loop is the reference implementation used for Windows development and the Pi service. The runtime configuration is kept in one YAML file so the audio rate, speech thresholds, model choices, endpoint, tool budget, and deployment path are not duplicated across scripts.

The current engineering scope includes interruptible spoken conversation, remote Qwen inference, project-local coding tools, public web lookup, timers, system information, and the existing ME2 bridge. It does not claim unrestricted Onit equivalence. An always-listening device needs a narrower default action surface than an interactive coding IDE. Capabilities that would add external side effects, credential use, repository rewrites, or arbitrary shell access must be added as reviewed typed tools.

3 Architecture

3.1 Audio and conversation path

The audio path is:

```
microphone -> Silero VAD -> faster-whisper -> Qwen
              -> Piper -> speaker.
```

Capture remains active while TTS plays. The local state machine has four states: `idle`, `listening`, `thinking`, and `speaking`. Speech during `speaking` stops playback and starts a new turn. A short pre-roll is retained so the beginning of an utterance is not clipped. The VAD endpoint requires a minimum speech interval followed by configured trailing silence.

The duplex state machine is independent of the Qwen model. This makes the interaction control testable without a

Table 1: Direct Qwen tools in the ME3 registry

Tool	Purpose
`list_files`	List project files and sizes.
`read_file`	Read bounded project text.
`write_file`	Create or replace a project text file.
`search_text`	Search project text files.
`run_command`	Run a direct allowlisted command.
`start_background_task`	Start a bounded build or test.
`task_status/output/cancel`	Supervise a background task.
`system_info`	Return non-secret host information.
`fetch_url/web_search`	Read public HTTP(S) resources.
`git_status`	Inspect repository status without mutation.

network request and leaves room for a Pipecat transport to reuse the same transition logic.

3.2 Tool-using Qwen brain

The remote brain is an OpenAI-compatible chat-completions client. The first request contains the recent dialogue and JSON Schema definitions for the ME3 tools. Qwen may return assistant text or a list of function calls. For every call, ME3 parses the arguments, executes the local function, appends a `role=tool` message, and asks Qwen to continue. The maximum number of tool rounds is eight by default; the checked-in configuration uses ten.

The protocol is illustrated below.

```
{
  "model": "Qwen/Qwen3.8-27B",
  "messages": [{"role":"user", "content":"build a web app"}],
  "tools": [{"type":"function", "function":
    {"name":"write_file", "parameters": {}}}],
  "tool_choice": "auto"
}
```

After a tool call, the result is returned as structured JSON. Qwen therefore receives the actual file path, byte count, return code, or error. The final spoken response is not accepted as evidence by itself.

3.3 Owned MCP services

The VAD server exposes frame and stream detection plus endpointing. The ASR server exposes PCM and file transcription. The TTS server exposes synthesis, streaming chunks, and installed voices. The tasks server retains timers, system information, and the ME2 bridge, and now wraps the same bounded project tool registry used by the direct Qwen path. The shared implementation avoids one policy for the local loop and another policy for MCP clients.

4 Tool Registry and Safety Boundary

Table 1 summarizes the direct functions visible to Qwen.

The path policy resolves every path inside the ME3 project. Credential-like names, environment files, private keys, the Git directory, the virtual environment, model caches, and generated release trees are protected. Reads and writes are size bounded. Commands are tokenized and run without a shell; shell operators and patterns associated with deletion, privilege escalation, SSH/SCP, repository reset/clean/push, and package publication are rejected. Long commands run with standard input closed, captured output, and a timeout.

This boundary is a design choice rather than an implementation accident. A voice transcript can be ambiguous, and the assistant is not an appropriate place for silent arbitrary shell execution. The registry is sufficient for the demonstration web app while keeping higher-risk operations explicit.

5 Web-App Capability

The target workflow is a dependency-free web application when the user does not name a framework. Qwen writes an application directory under `agent_workspace/<name>`, lists the result, reads the important files, and runs a bounded verification command if one is appropriate. The provided demonstration contains `index.html`, `styles.css`, and `app.js`; it provides a command input, microphone placeholder, status area, and tool-activity log.

The capability is demonstrated twice: the first live request used Qwen to create the HTML file through a real tool call; a later live request used Qwen to list the directory and read all three files. This demonstrates inference, tool dispatch, file mutation, and evidence retrieval without training a model. The demo is intentionally local and static; the current safety policy does not publish it to the public internet or install an arbitrary server.

6 Deployment Workflow

ME3 follows a four-stage local-dev workflow:

(1) **Environment gate:** import the shared Python dependencies.
(2) **Smoke gate:** load all four MCP services, call their health tools, construct the engineering benchmark, and query PortAudio when available.
(3) **Package:** copy the runtime files into a versioned release, write SHA-256 entries, and create a zip archive.
(4) **Deploy:** transfer the archive to the configured Pi, merge its stable `me3_voice_duplex` directory, run the idempotent installer, and record a deployment receipt.

The current target is dalmacio@cfdfnjrpi5.local at /home/dalmacio/Desktop/kiko/me3_voice_duplex. The separate ME2 deployment remains at /home/dalmacio/Desktop/dandan. A prepared release is not called deployed. The local receipt must say `deployed` after SSH, SCP, extraction, and installation succeed. The current receipt reports `deployed` for release `me3-20261024-173332-36bb26b8`; after installation, a `--check` pass and a text-mode Qwen run completed on the Pi that built a static stopwatch web app and served it over HTTP, with the page cross-verified from a second machine on the LAN.

The API key is never included in the archive. On the Pi it must be provisioned through a secure runtime environment. The Windows DPAPI helper can report configured/not-configured and can provide an in-memory child-process value

for local testing, but its secret is not printed or written to the repository.

7 Verification and Evidence

The current local verification sequence is:

```
python scripts/dev_env_check.py
python scripts/local_smoke_test.py --with-audio
python scripts/check_paper_alignment.py
python -m pytest tests/test_tooling.py -q
python -m me3.agent --brain remote --text \\
  "Use system_info and summarize the host."
```

The tool-focused tests cover OpenAI-compatible schemas, blocked shell patterns, background task completion, and a mocked multi-step Qwen response. The live Qwen smoke test records an actual tool call and final answer. The local smoke gate loads the four MCP servers and confirms the current Windows PortAudio inventory. These are engineering checks, not a claim about Pi audio.

The project also contains a separate engineering latency driver and a paper-native FULL-DUPLEX-BENCH v1.5 contract. The v1.5 contract includes four scenarios with 200, 99, 100, and 100 stimuli and response categories RESPOND, RESUME, UNCERTAIN, and UNKNOWN. The engineering five-scenario driver must not be presented as the paper's 499-utterance result. A complete captured overlap artifact, transcript classification, and prosodic before/after analysis are still required before claiming a paper-grade result.

8 Discussion

The main engineering trade-off is locality versus model capability. Keeping speech I/O local reduces the need to upload audio and makes the Pi responsive, while Qwen supplies stronger reasoning and code generation than a small local model. The cost is network dependence, endpoint latency, and the need to provision the API key on the target device. The typed registry makes the boundary visible and testable, but it is not a complete replacement for an interactive coding environment.

The full-duplex behavior is also more than a transport setting. VAD must keep running during playback, the audio output must be cancellable, and the turn-state transition must preserve the interrupted user's utterance. Echo control, microphone placement, and ALSA device selection remain physical Pi concerns. A successful Windows loopback does not prove those conditions.

Future work should add a Pi-local live self-test, secure credential provisioning guidance, a reviewed web-app preview server with explicit approval, and the full v1.5 overlap capture/evaluation. Additional Onit-like capabilities should be introduced as typed, auditable functions rather than by removing the voice safety boundary.

9 Conclusion

ME3 now combines full-duplex voice interaction with a real Qwen tool loop. A spoken build request can result in actual project files and verification, and the agent can supervise longer work without freezing the microphone. The implementation uses inference and tools rather than model training, keeps local speech services under project ownership, and records the distinction between local verification, prepared release, Pi deployment, and benchmark evidence. On the deployed Pi, a securely configured Qwen runtime key and a text-mode Qwen tool run are verified; the next decisive checks are a live Pi run with physical audio and a clean, captured v1.5 overlap evaluation.

References

[1] Lin et al. "FULL-DUPLEX-BENCH v1.5: Evaluating Overlap." Project source PDF: *FULL-DUPLEX-BENCH V1.5 - EVALUATING OVERLAP.pdf*, AI 231 ME3 repository, 2026.

[2] "Building AI Voice Agents - Summary." Project source PDF, AI 231 ME3 repository, 2026.

[3] A. Radford et al. 2023. Robust speech recognition via large-scale weak supervision. In *Proceedings of ICML*.

[4] Silero Team. Silero VAD. Open-source voice activity detection model and runtime. The ME3 release pins the installed runtime version in `requirements.txt`.

[5] Model Context Protocol. Official protocol specification and SDK. The ME3 release uses the Python MCP/FastMCP stack for owned task services.

[6] Pipecat. Open-source framework for real-time voice and multimodal agents. The ME3 release pins Pipecat 1.13.0 for the harness-facing adapter.